

PROJETO 4 – Tarefa 1 – Lista de Exercícios

Gustavo Borges Peres da Silva¹

¹Departamento de Ciência da Computação – Universidade Federal de Catalão (UFCAT)

gustavo.silva@discente.ufcat.edu.br

Informação Administrativa: para a resolução desta lista, considera-se que o aluno possua uma distribuição MINIX funcional e que esteja realizando a prática de laboratório.

A LISTA DEVE SER ENTREGUE VIA SIGAA ATÉ 01/02/2023.

Objetivos

- Familiarizar o aluno com o SO MINIX.

Perguntas específicas envolvendo o conteúdo do slide estudado sobre MINIX:

- 1) Considerando o MINIX, como as funções do sistema são distribuídas em espaço de kernel e espaço de usuário?

No MINIX, as funções do sistema são distribuídas entre o espaço de kernel e o espaço de usuário. O espaço de usuário atua como um sandbox, impedindo que programas causem danos à memória e a outros recursos do sistema. O kernel, que tem acesso total à memória e ao hardware, executa em espaço de kernel. Processos de usuário executam em espaço de usuário e, para interagir com o kernel, precisam usar chamadas de sistema como open, read, write, etc.

- 2) Qual é o papel da TAREFA de sistema e de clock?

No modo kernel do MINIX, existem duas tarefas principais: a TAREFA de sistema, responsável por atender chamadas de sistema, e a tarefa de clock, um dispositivo de I/O que interage com o hardware gerador de sinais de temporização. A tarefa de clock não é acessível diretamente pelos usuários, ao contrário de outros dispositivos que são acessados por meio da TAREFA de sistema.

- 3) Qual é a relação de privilégios entre as camadas 1 (kernel), 2 (device drivers), 3 (servidores) e 4 (processos de usuário) no MINIX?

As camadas no MINIX são organizadas hierarquicamente em termos de privilégios, sendo que o privilégio diminui à medida que a camada aumenta:

- **Camada 1 (Kernel):** A camada mais privilegiada, com acesso total à memória e hardware do sistema. Realiza operações críticas e essenciais para o funcionamento do sistema.
- **Camada 2 (Device Drivers):** Situada abaixo do kernel, essa camada contém os drivers de dispositivos. Os drivers realizam operações específicas para interagir com hardware, mas com um nível de privilégio inferior ao do kernel.

- **Camada 3 (Servidores):** Responsável por fornecer serviços específicos, como gerenciamento de arquivos e sistema de janelas. Possui menos privilégios do que as camadas anteriores, mas mais do que os processos de usuário.
- **Camada 4 (Processos de Usuário):** A camada menos privilegiada, composta pelos processos de usuário normais. Eles têm acesso limitado e controlado às funcionalidades do sistema operacional, garantindo segurança e estabilidade.

Essa estrutura hierárquica visa garantir a segurança e a estabilidade do sistema, limitando o acesso aos recursos conforme necessário.

- 4) Forneça 3 exemplos de chamadas de sistema no MINIX, incluindo a descrição funcional e a especificação dos parâmetros de entrada e saída.
 - **fork():** Cria um novo processo duplicando o processo existente.
 - **read(fd, buffer, nbytes):** Lê dados de um arquivo ou dispositivo.
 - **write(fd, buffer, nbytes):** Escreve dados em um arquivo ou dispositivo.

- 5) Diferencie chamada de sistema (system call) de chamada de kernel (kernel call) no MINIX.

Chamadas de kernel são funções de baixo nível fornecidas pela TAREFA de sistema para drivers e servidores. Exemplo: leitura de porta de I/O. Chamadas de sistema (ex. read, fork, unlink) são de alto nível, definidas pelo padrão POSIX, disponíveis para programas de usuário no nível 4.

- 6) Considere o seguinte exemplo: um arquivo no MINIX possui um proprietário com o uid = 12 e gid = 1 e tem o modo de acesso rwxr-x---. Explique o significado deste modo de acesso. Se outro usuário com o uid = 6 e gid = 1 tenta executar o arquivo, o que irá acontecer?

No exemplo rwxr-x---, o proprietário (uid=12) tem permissão total, o grupo (gid=1) tem permissão de leitura e execução, e outros não têm permissão. O usuário com uid=6 e gid=1 terá permissão de leitura e execução, herdada do grupo. Portanto, ele poderá ler o arquivo.

- 7) Em vista do fato de que a mera existência de um super-usuário possa levar a todos os tipos de problemas de segurança, por que tal conceito existe?

O superusuário existe para enfatizar a política de separação de privilégios. É usado principalmente para tarefas de administração e é permitido apenas a usuários autorizados. O processo init, executado como root, carrega outros processos e herda a prioridade para garantir a execução controlada.

- 8) No MINIX, se o usuário 2 cria um link para um arquivo de um usuário 1, e então o usuário 1 remove este arquivo, o que acontece quando o usuário 2 tenta ler o arquivo? Explique (Dica: simule este comportamento. Use para tal o comando ln).

Se o usuário 2 cria um link para um arquivo do usuário 1 e o usuário 1 remove o arquivo, o link criado pelo usuário 2 se torna inválido, e ele não pode mais ler o arquivo.

- 9) Como o reincarnation server (RS) opera com relação à manutenção dos controladores de dispositivo no MINIX 3?

O RS periodicamente verifica os controladores de dispositivo. Se um driver não responde corretamente, o RS substitui automaticamente por uma nova cópia, garantindo a recuperação automática de falhas.

- 10) Considerando os Sistemas Operacionais (SOs) tradicionais (ex. Windows, OS X, Linux etc), quais deles possuem características semelhantes ao serviço fornecido pelo RS no MINIX?

Em sistemas tradicionais como Windows Server, OS X e Red Hat Enterprise Linux, existem mecanismos semelhantes para lidar com a recuperação de falhas e atualizações dinâmicas de drivers.

- 11) Por que o MINIX mantém o programa update em execução, em segundo plano, todo o tempo?

O programa "update" é mantido em execução em segundo plano para emitir chamadas de sistema SYNC a cada 30 segundos. Isso garante que as modificações no sistema sejam escritas no disco, contribuindo para a estabilidade do sistema e evitando problemas durante a parada ou execução de programas críticos.

- 12) Considere a seguinte estrutura de processos.

PID TTY TIME CMD

(-4) ? 0:29 idle

(-3) ? 0:00 clock

(-2) ? 0:00 system

(-1) ? 0:00 kernel

(0) ? 0:00 pm

(1) ? 0:00 init

Explique a relação entre os índices de PID do init e demais processos apresentados.

Os índices negativos representam tarefas do sistema no MINIX. A árvore de processos é formada por todos os processos de sistema e de usuário, e o init (PID 1) é o processo pai para muitos outros processos. A localização dos índices negativos não está na raiz da árvore, mas segue a hierarquia estabelecida durante a execução do projeto 3.

- 13) Considere as chamadas `fork()` e `pthread_create()`. Qual é o nível de conhecimento do kernel do MINIX para cada uma destas chamadas? No caso de threads, como se classificariam as threads implementadas no MINIX 3?~

No MINIX, o kernel possui conhecimento total da chamada de sistema `fork()`, pois esta envolve a criação de um novo processo. Já a chamada `pthread_create()` não possui contra parte em espaço de kernel e é tratada inteiramente em espaço de usuário, classificando as threads como threads de espaço de usuário no MINIX 3.

- 14) Explique como um processo realiza uma chamada a um device driver.

Um processo realiza uma chamada a um device driver enviando mensagens ao servidor apropriado, como o servidor de sistema de arquivos (FS). O servidor, por sua vez, envia uma mensagem ao device driver para executar operações de leitura ou escrita. Chamadas de kernel são usadas para coordenar o processo de I/O e copiar dados entre os processos.

- 15) Com o conhecimento adquirido e com base no material de apoio, faça um breve comparativo entre o SO MINIX e o SO UNIX.

O MINIX utiliza um kernel microkernel, enquanto o UNIX tradicionalmente possui um kernel monolítico. Ambos suportam chamadas de sistema e possuem características de sistemas de arquivos semelhantes. No entanto, o MINIX destaca-se por sua abordagem modular e ênfase na simplicidade e educação.

- 16) Qual é o papel do System Event Framework (SEF) em um device driver?

O SEF no MINIX permite que desenvolvedores registrem chamadas de tratamento (callbacks) para eventos como inicialização, atualização e reinicialização de serviços. Ele mascara os detalhes de implementação do RS e facilita a interação entre o kernel e os drivers.

Perguntas específicas envolvendo o uso do sistema operacional MINIX instalado:

- 17) Por que uma instalação recente do MINIX viria com um usuário existente chamado "ast"?

O usuário "ast" é incluído como um exemplo de usuário no MINIX, talvez como uma homenagem ao autor do livro texto, Andrew Stuart Tanenbaum.

- 18) Qual é o nome da aplicação (utilitário) que você usou para instalar novos softwares no MINIX? Dê exemplos de uso.

O utilitário usado para instalar novos softwares no MINIX é o `pkgin`. Exemplos de uso incluem comandos como `pkgin list`, `pkgin avail` e `pkgin update`.

- 19) Qual é o comando que você deve usar para refazer o kernel (build) e instalá-lo?

O comando para refazer e instalar o kernel no MINIX pode ser `make world` e `make install` em `/usr/src`.

20) Qual é o arquivo fonte que possui a string do banner do MINIX (i.e. “MINIX 3.1.X ... Copyright ...”) que é mostrado quando o MINIX inicializa?

O arquivo fonte que possui a string do banner do MINIX é `usr/src/kernel/main.c`.

21) Qual é o nome da chamada de sistema do MINIX com o número 33 (decimal), e como você determinou esta informação?

A chamada de sistema do MINIX com o número 33 é a `access`, conforme encontrado no arquivo `/usr/src/servers/pm/table.c`. Essa informação foi determinada por meio de pesquisa online relacionada às chamadas de sistema no MINIX.